

// Jivops

Guía completa — 30 lecciones

RBAC · Escalado · Networking · Observabilidad · GitOps · Resiliencia

Contenido

Módulo 1: Seguridad y Control de Acceso

- Lección 01 — Introducción a RBAC
- Lección 02 — Roles y RoleBindings
- Lección 03 — ClusterRoles y ClusterRoleBindings
- Lección 04 — ServiceAccounts
- Lección 05 — Network Policies — fundamentos
- Lección 06 — Network Policies — reglas avanzadas

Módulo 2: Escalado y Recursos

- Lección 07 — Resource Requests & Limits
- Lección 08 — Horizontal Pod Autoscaler (HPA)
- Lección 09 — Vertical Pod Autoscaler (VPA)
- Lección 10 — Pod Disruption Budgets
- Lección 11 — Node Affinity y Anti-Affinity
- Lección 12 — Taints y Tolerations

Módulo 3: Networking y Exposición

- Lección 13 — Ingress Controllers
- Lección 14 — Ingress con TLS y cert-manager
- Lección 15 — Service Mesh — introducción
- Lección 16 — DNS interno y CoreDNS
- Lección 17 — Load Balancing avanzado

Módulo 4: Observabilidad en Producción

- Lección 18 — Liveness y Readiness Probes avanzados
- Lección 19 — Logging centralizado
- Lección 20 — Métricas custom con Prometheus
- Lección 21 — Alerting con Alertmanager
- Lección 22 — Distributed Tracing

Módulo 5: GitOps y Despliegue

- Lección 23 — Introducción a GitOps
- Lección 24 — ArgoCD — instalación y sync
- Lección 25 — Estrategias de despliegue (Blue-Green, Canary)
- Lección 26 — Rollbacks y versionado

Módulo 6: Resiliencia y Seguridad Operacional

Lección 27 — Backups con Velero

Lección 28 — Disaster Recovery

Lección 29 — Políticas de seguridad con Kyverno

Lección 30 — Proyecto final: app completa en producción

Módulo 1: Seguridad y Control de Acceso

Lección 01 — Introducción a RBAC

RBAC (Role-Based Access Control) controla qué puede hacer cada usuario, grupo o ServiceAccount dentro del cluster. Se compone de cuatro objetos: Role, ClusterRole, RoleBinding y ClusterRoleBinding.

Un Role define permisos dentro de un namespace; un ClusterRole aplica a todo el cluster o a recursos sin namespace (como nodos).

```
kubectl auth can-i create pods --namespace dev
kubectl get roles,rolebindings -n dev
```

Lección 02 — Roles y RoleBindings

Un Role especifica verbos (get, list, create, delete...) sobre recursos concretos (pods, deployments, secrets). Un RoleBinding conecta ese Role con un usuario, grupo o ServiceAccount dentro del namespace.

```
kubectl create role pod-reader --verb=get,list --resource=pods -n dev
kubectl create rolebinding read-pods --role=pod-reader --user=carlos -n dev
```

Lección 03 — ClusterRoles y ClusterRoleBindings

Cuando el permiso debe aplicar a todos los namespaces o a recursos cluster-wide (nodos, persistentvolumes, namespaces), se usa ClusterRole. Se puede vincular con RoleBinding (limitado a un namespace) o ClusterRoleBinding (todo el cluster).

```
kubectl create clusterrole node-viewer --verb=get,list --resource=nodes
kubectl create clusterrolebinding view-nodes --clusterrole=node-viewer --user=carlos
```

Lección 04 — ServiceAccounts

Cada Pod se ejecuta con una identidad: su ServiceAccount. Por defecto usa "default", pero en producción cada aplicación debe tener su propio ServiceAccount con permisos mínimos (principio de menor privilegio).

```
kubectl create serviceaccount app-sa -n prod
kubectl create rolebinding app-binding --role=app-role --serviceaccount=prod:app-sa -n prod
```

Lección 05 — Network Policies — fundamentos

Por defecto, todos los Pods de un cluster pueden hablar entre sí sin restricción. Las NetworkPolicy permiten definir reglas de firewall a nivel de Pod: qué tráfico entra (ingress) y qué tráfico sale (egress).

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
```

Lección 06 — Network Policies — reglas avanzadas

Una vez definido un "deny all" por defecto, se abren excepciones específicas: solo el frontend puede hablar con el backend, solo el backend puede hablar con la base de datos, etc. Esto limita el radio de impacto

ante una brecha de seguridad.

```
spec:
  podSelector:
    matchLabels: {app: backend}
  ingress:
    - from:
      - podSelector:
          matchLabels: {app: frontend}
```

Módulo 2: Escalado y Recursos

Lección 07 — Resource Requests & Limits

"requests" es lo que el Pod garantiza reservar; "limits" es el techo máximo que puede consumir. Sin estos valores, un solo Pod puede consumir toda la memoria del nodo y tumbar otros servicios (noisy neighbor).

```
resources:
  requests: {cpu: "250m", memory: "256Mi"}
  limits: {cpu: "500m", memory: "512Mi"}
```

Lección 08 — Horizontal Pod Autoscaler (HPA)

El HPA aumenta o reduce el número de réplicas de un Deployment según métricas como uso de CPU o memoria, manteniendo el servicio estable ante picos de tráfico sin intervención manual.

```
kubectl autoscale deployment api --cpu-percent=70 --min=2 --max=10
kubectl get hpa
```

Lección 09 — Vertical Pod Autoscaler (VPA)

Mientras el HPA cambia el número de réplicas, el VPA ajusta los requests/limits de cada Pod según su consumo real histórico. Útil para cargas con un solo Pod que no pueden escalar horizontalmente.

Lección 10 — Pod Disruption Budgets

Un PDB garantiza un número mínimo de réplicas disponibles durante operaciones voluntarias (drenar un nodo, actualizar el cluster), evitando que Kubernetes tumbe demasiados Pods a la vez.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
spec:
  minAvailable: 2
  selector:
    matchLabels: {app: api}
```

Lección 11 — Node Affinity y Anti-Affinity

Permiten controlar en qué nodos se programan los Pods (por ejemplo, nodos con GPU) o forzar que réplicas de una misma app no compartan nodo, mejorando la tolerancia a fallos.

Lección 12 — Taints y Tolerations

Un taint en un nodo repele Pods por defecto; solo los Pods con la toleration correspondiente pueden programarse ahí. Se usa para reservar nodos dedicados (ej. solo cargas de base de datos).

```
kubectl taint nodes node1 dedicated=db:NoSchedule
```

Módulo 3: Networking y Exposición

Lección 13 — Ingress Controllers

Un Ingress Controller (nginx-ingress, Traefik) enruta tráfico HTTP/HTTPS externo hacia los Services internos según host o path, evitando exponer un LoadBalancer por cada servicio.

Lección 14 — Ingress con TLS y cert-manager

cert-manager automatiza la emisión y renovación de certificados TLS (Let's Encrypt) para los recursos Ingress, evitando la gestión manual de certificados en producción.

```
annotations:
  cert-manager.io/cluster-issuer: letsencrypt-prod
tls:
  - hosts: [api.midominio.com]
    secretName: api-tls
```

Lección 15 — Service Mesh — introducción

Un service mesh (Istio, Linkerd) añade un proxy sidecar a cada Pod para gestionar tráfico, seguridad (mTLS) y observabilidad entre servicios sin cambiar el código de la aplicación.

Lección 16 — DNS interno y CoreDNS

CoreDNS resuelve nombres de Service dentro del cluster (servicio.namespace.svc.cluster.local), permitiendo que los Pods se comuniquen entre sí por nombre en vez de IP.

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
```

Lección 17 — Load Balancing avanzado

Más allá del balanceo básico de un Service, se puede ajustar la afinidad de sesión, algoritmos de balanceo y health checks activos para tráfico de producción de alto volumen.

Módulo 4: Observabilidad en Producción

Lección 18 — Liveness y Readiness Probes avanzados

Un liveness probe mal configurado reinicia Pods sanos bajo carga; un readiness probe débil deja recibir tráfico a un Pod que aún no está listo. En producción se ajustan thresholds, timeouts y delays con cuidado.

```
readinessProbe:
  httpGet: {path: /health, port: 8080}
  initialDelaySeconds: 5
  periodSeconds: 10
  failureThreshold: 3
```

Lección 19 — Logging centralizado

En producción los logs no viven en el nodo: se envían a un stack centralizado (Loki, ELK) vía un agente (Fluent Bit, Promtail) para poder buscarlos y correlacionarlos entre Pods.

Lección 20 — Métricas custom con Prometheus

Además de las métricas de infraestructura, las aplicaciones pueden exponer métricas de negocio propias (pedidos procesados, errores por endpoint) en formato Prometheus para que sean scrapeadas.

```
# En la app: exponer /metrics
# En Prometheus:
scrape_configs:
  - job_name: 'api'
    static_configs:
      - targets: ['api:8080']
```

Lección 21 — Alerting con Alertmanager

Alertmanager recibe alertas de Prometheus, las agrupa, evita duplicados y las enruta (Slack, email, PagerDuty) según severidad, evitando fatiga de alertas en el equipo.

Lección 22 — Distributed Tracing

En una arquitectura de microservicios, una petición puede pasar por varios servicios. El tracing distribuido (Tempo, Jaeger) permite seguir esa petición de punta a punta e identificar dónde se pierde tiempo.

Módulo 5: GitOps y Despliegue

Lección 23 — Introducción a GitOps

GitOps trata el repositorio Git como única fuente de verdad del estado del cluster: en vez de ejecutar `kubectl apply` manualmente, un controlador sincroniza automáticamente lo que hay en Git.

Lección 24 — ArgoCD — instalación y sync

ArgoCD compara continuamente el estado deseado (en Git) con el estado real del cluster, y puede aplicar los cambios automáticamente o dejarlos pendientes de aprobación manual.

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

Lección 25 — Estrategias de despliegue (Blue-Green, Canary)

Blue-Green mantiene dos versiones completas y cambia el tráfico de una vez; Canary desplaza tráfico gradualmente a la nueva versión, reduciendo el riesgo de un despliegue fallido en producción.

Lección 26 — Rollbacks y versionado

Cada despliegue debe poder revertirse rápido. Kubernetes guarda un historial de revisiones de cada Deployment que permite volver atrás en segundos ante un fallo detectado tras el despliegue.

```
kubectl rollout history deployment/api
kubectl rollout undo deployment/api --to-revision=3
```

Módulo 6: Resiliencia y Seguridad Operacional

Lección 27 — Backups con Velero

Velero permite hacer backup de recursos del cluster y volúmenes persistentes, y restaurarlos ante un desastre o para clonar un entorno completo a otro cluster.

```
velero backup create backup-prod --include-namespaces prod
velero restore create --from-backup backup-prod
```

Lección 28 — Disaster Recovery

Un plan de DR define objetivos de tiempo de recuperación (RTO) y punto de recuperación (RPO), y se prueba periódicamente — un backup que nunca se restauró no es un plan de recuperación real.

Lección 29 — Políticas de seguridad con Kyverno

Kyverno aplica políticas como código: bloquea el despliegue de imágenes sin firmar, exige límites de recursos definidos, o prohíbe contenedores corriendo como root, antes de que lleguen al cluster.

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
spec:
  rules:
    - name: require-limits
      validate:
        message: "Se requieren resource limits"
```

Lección 30 — Proyecto final: app completa en producción

Se integra todo lo anterior: una aplicación desplegada vía ArgoCD, con RBAC y Network Policies aplicados, HPA activo, Ingress con TLS, métricas en Grafana, alertas configuradas y backups programados con Velero. Este es el nivel que se espera de un DevOps/SRE en producción real.